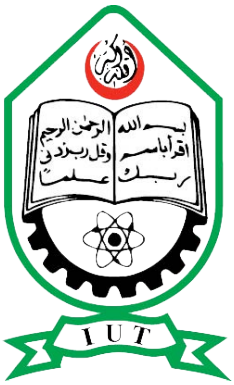

(CSE 4802: Compiler Design Lab)

LAB 1

Report on Lexical Analyzer

Submitted by:
Md. Mosharrifur Rahman Ratul
Student ID: 210041213
Section: 2A
BSc in CSE, Dept of Computer Science and Engineering
Islamic University of Technology (IUT)

May 1, 2026

Contents

1 Introduction

A lexical analyzer is the first phase of a compiler. Its main job is to read the source code as a stream of characters and convert it into meaningful units called tokens. This process helps the compiler understand the structure of the program before further analysis.

1.1 Key Concepts

Some of the key concepts regarding the lexical analyzer are:-

- **Token:** A token is a basic unit of the program defining a syntactic category, such as a keyword, identifier, operator, etc.
- **Lexeme:** A lexeme is the actual sequence of characters in the source code that matches a token. For example, in `int x = 5;`, `x` is a lexeme.
- **Pattern:** A pattern is a rule, often defined using regular expressions, that describes the form of a token or used to identify a token.

2 Problem Statement

Design and implement a lexical analyzer for a subset programming language. The analyzer should:

- Read a program written in the defined subset language.
- Identify and classify tokens
- Generate the following outputs:
  - Token stream
  - Token statistics
  - Symbol table
- Detect and report errors

3 Code Implementation

The lexical analyzer is implemented in C++. The following code snippet shows the main implementation parts:

Listing 1: Predefined arrays for categorizing

```
    const unordered_set<string> keywords = {
        "int", "float", "double", "char", "bool", "void",
        "if", "else", "for", "while", "do",
        "switch", "case", "break", "continue", "return",
        "class", "struct", "public", "private", "using", "
            namespace", "nullptr"
    };
    const unordered_set<string> opIncDec = {"++", "--"};
    const unordered_set<string> opAssignment = {
        "=", "+=", "-=", "*=", "/=", "%="
    };
    const unordered_set<string> opArithmetic = {"+", "-", "*",
        "/", "%"};
    const unordered_set<string> opRelational = {"==", "!=", "<"
        , ">", "<=", ">="};
    const unordered_set<string> opLogical = {"&&", "||", "!"};
    const unordered_set<string> opBitwise = {"&", "|", "^", "~"
        , "<<", ">>"};

    const unordered_set<string> delimiters = {
        ";", ",", "(", ")", "{", "}", "[", "]", ".", ":", "#"
    };
    const vector<string> multiOps = {
        "++", "--",
        "+=", "-=", "*=", "/=", "%=",
        "==", "!=", "<=", ">=",
        "&&", "||",
        "<<", ">>"
    };
```

Listing 2: Code for comment identification

```
        string trimmed = ltrim(line);

        if (inBlockComment) {
            currentComment += "\n" + line;
            if (trimmed.find("*/") != string::npos || line.find
                ("*/") != string::npos) {
                commentBlocks.push_back(currentComment);
                currentComment.clear();
                inBlockComment = false;
            }
            continue;
```

```
        }

        if (trimmed.rfind("//", 0) == 0) {
            commentBlocks.push_back(line);
            continue;
        }
        if (trimmed.rfind("/*", 0) == 0) {
            currentComment = line;
            if (line.find("*/") != string::npos) {
                commentBlocks.push_back(currentComment);
                currentComment.clear();
            } else {
                inBlockComment = true;
            }
            continue;
        }
```

Listing 3: Code for Token Categorization

```
            bool matchedMulti = false;
            for (const string& op : multiOps) {
                if (isPreprocessorLine && (op.find('<') !=
                    string::npos || op.find('>') != string::npos
                    )) {
                    continue;
                }
                if (i + op.size() <= n && line.compare(i, op.
                    size(), op) == 0) {
                    addOperator(op);
                    i += op.size();
                    matchedMulti = true;
                    break;
                }
            }
            if (matchedMulti) continue;

            string one(1, c);
            if (isPreprocessorLine && (c == '<' || c == '>')) {
                lineDelimiters.push_back(one);
                tokenStream.push_back(one);
                typedStream.push_back({"DELIMITER", one});
                i++;
                continue;
            }
            if (delimiters.count(one)) {
                lineDelimiters.push_back(one);
                tokenStream.push_back(one);
                typedStream.push_back({"DELIMITER", one});
                i++;
                continue;
```

```
            }
            if (opArithmetic.count(one) || opRelational.count(one) || opLogical.count(one) || opBitwise.count(one) || opAssignment.count(one)) {
                addOperator(one);
                i++;
                continue;
            }

            if (isIdentifierStart(c)) {
                size_t start = i;
                i++;
                while (i < n && isIdentifierChar(line[i])) i++;
                string word = line.substr(start, i - start);
                tokenStream.push_back(word);
                if (keywords.count(word)) {
                    lineKeywords.push_back(word);
                    typedStream.push_back({"KEYWORD", word});
                } else {
                    lineIdentifiers.push_back(word);
                    typedStream.push_back({"IDENTIFIER", word});
                    identifierFreq[word]++;
                    if (!identifierFirstLine.count(word))
                        identifierFirstLine[word] = lineNo;
                }
                continue;
            }
```

Listing 4: Code for Error identification

```
            while (i < n) {
                char ch = line[i];
                if (std::isdigit(static_cast<unsigned char>(ch))) {
                    if (seenExp) expDigits = true;
                    i++;
                    continue;
                }
                if (ch == '.' && !seenDot && !seenExp) {
                    seenDot = true;
                    i++;
                    continue;
                }
                if (ch == '.' && (seenDot || seenExp)) {
                    invalidNumeric = true;
                    break;
                }
                if ((ch == 'e' || ch == 'E') && !seenExp) {
```

```
                    seenExp = true;
                    i++;
                    if (i < n && (line[i] == '+' || line[i]
                        == '-')) i++;
                    continue;
                }
                if ((ch == 'e' || ch == 'E') && seenExp) {
                    invalidNumeric = true;
                    break;
                }
                break;
            }

            if (invalidNumeric) {
                size_t j = i;
                while (j < n && !isBoundaryChar(line[j])) j
                    ++;
                string bad = line.substr(start, j - start);
                addError("Invalid numeric format", bad);
                i = j;
                continue;
            }

            if (seenExp && !expDigits) {
                string bad = line.substr(start, i - start);
                addError("Invalid numeric format", bad);
                continue;
            }

            if (i < n && isIdentifierChar(line[i])) {
                size_t j = i;
                while (j < n && isIdentifierChar(line[j]))
                    j++;
                string bad = line.substr(start, j - start);
                addError("Invalid identifier", bad);
                i = j;
                continue;
            }

        if (isPunct(c)) {
            addError("Unknown symbol", one);
            i++;
            continue;
        }

        if (!isStopChar(c)) {
            string unk(1, c);
            addError("Unknown symbol", unk);
        }
        i++;
```

4 Approach

The lexical analyzer defines different sets of tokens such as keywords, operators, and delimiters. It scans each line to find the token stream and store token statistics, errors, comments, and symbol table information.

For each line of the input file, the following steps are performed:

1. **Preprocessing for comments:**
   The line is trimmed to remove leading spaces. The analyzer first checks for comments. Single-line comments (`//`) and multi-line comments (`/* */`) are detected and stored separately without further processing.

2. **Token Scanning:**
   The line is scanned from left to right using an index pointer. Each character is examined to determine the type of token.

3. **String and Character Constants:**
   If a double quote (`"`) or single quote (`'`) is found, the analyzer extracts the complete string or character constant, including escape sequences.

4. **Operators Detection:**
   The analyzer first checks for multi-character operators (like ==, ++, +=). If not found, it checks for single-character operators and classifies them into arithmetic, relational, logical, bitwise, assignment, or increment/decrement operators. These are predefined in the respective arrays, the analyzer checks for the match and classifies them according to it.

5. **Delimiters Identification:**
   Symbols such as parentheses, braces, commas, and semicolons are identified as delimiters similarly from the predefined array.

6. **Identifiers and Keywords:**
   If a valid starting character (letter or underscore) is found, the analyzer extracts the full word. It then checks whether the word is a keyword from a predefined array of keyword for the

language. If it is not a keyword , then this is categorized as an identifier. Identifiers are stored along with their frequency and first occurrence line number.

7. **Numeric Constants:** Numbers are processed to distinguish between integer and floating-point constants. The analyzer also handles scientific notation and detects invalid numeric formats.
8. **Error Handling:** Invalid tokens such as malformed numbers, unknown symbols, or incorrect identifiers are detected and recorded with line numbers.
9. **Token Stream Generation:** For each valid token, a pair of token type and lexeme is generated and stored as part of the token stream.
10. **Statistics Update:** Counters for different types of tokens are updated, and unique tokens are tracked.

After processing all lines, the analyzer generates a summary including token stream, token statistics and creates a symbol table for identifier frequencies. It also creates an error summary and show the comments in the code.

5 Outputs

5.1 Input File

To evaluate the performance of the lexical analyzer, a sample C++ input file was used. This input includes valid tokens, comments as well as some intentional lexical errors (such as invalid identifiers and malformed numbers) to test the robustness of the analyzer.

Listing 5: Sample Input File for Lexical Analysis

```
1
2  #include<iostream>
3  #include<vector>
4  using namespace std;
5
6  int main(){
7      int t,a,b,@total,5pair;
8      cin >> t;
9
```

```
    int summation = a+b;
    int diff = a-b;
    int multiply = a*b;
    total = 11.5*a + 21.22.;

// initialize a vector

    vector<ll>a;
    for(ll i=0;i<a.size();i++)
        cout << a[i] << ' ';
    cout << endl;
}
```

5.2 Output Token Stream

For the given input file, for each line the output token stream is following:-

```

Line: #include<iostream>
Token detected: "#", "include", "<", "iostream", ">"
Findings : ----------------
Token Stream :
<  DELIMITER , #  >
<  IDENTIFIER , include  >
<  DELIMITER , <  >
<  IDENTIFIER , iostream  >
<  DELIMITER , >  >
----------------------------------------
----------------------------------------


Line: using namespace std;
Token detected: "using", "namespace", "std", ";"
Findings : ----------------
Token Stream :
<  KEYWORD , using  >
<  KEYWORD , namespace  >
<  IDENTIFIER , std  >
<  DELIMITER , ;  >
----------------------------------------
----------------------------------------


Line: int main(){
Token detected: "int", "main", "(", ")", "{"
Findings : ----------------
```

```
Token Stream :
<  KEYWORD , int  >
<  IDENTIFIER , main  >
<  DELIMITER , (  >
<  DELIMITER , )  >
<  DELIMITER , {  >
----------------------------------------
----------------------------------------


Line:     int t,a,b,@total,5pair;
Token detected: "int", "t", ",", "a", ",", "b", ",", "total",
    ",", ";"
Findings : ----------------
Token Stream :
<  KEYWORD , int  >
<  IDENTIFIER , t  >
<  DELIMITER , ,  >
<  IDENTIFIER , a  >
<  DELIMITER , ,  >
<  IDENTIFIER , b  >
<  DELIMITER , ,  >
<  IDENTIFIER , total  >
<  DELIMITER , ,  >
<  DELIMITER , ;  >
Error Findings : ----------------
Errors: [Unknown symbol: @, Invalid identifier: 5pair]
----------------------------------------
----------------------------------------


Line:     cin >> t;
Token detected: "cin", ">>", "t", ";"
Findings : ----------------
Token Stream :
<  IDENTIFIER , cin  >
<  BITWISE_OP , >>  >
<  IDENTIFIER , t  >
<  DELIMITER , ;  >
----------------------------------------
----------------------------------------


Line:     int summation = a+b;
Token detected: "int", "summation", "=", "a", "+", "b", ";"
Findings : ----------------
Token Stream :
<  KEYWORD , int  >
<  IDENTIFIER , summation  >
<  ASSIGNMENT_OP , =  >
```

```
<  IDENTIFIER , a  >
<  ARITHMETIC_OP , +  >
<  IDENTIFIER , b  >
<  DELIMITER , ;  >
----------------------------------------
----------------------------------------


Line:     int diff = a-b;
Token detected: "int", "diff", "=", "a", "-", "b", ";"
Findings : ----------------
Token Stream :
<  KEYWORD , int  >
<  IDENTIFIER , diff  >
<  ASSIGNMENT_OP , =  >
<  IDENTIFIER , a  >
<  ARITHMETIC_OP , -  >
<  IDENTIFIER , b  >
<  DELIMITER , ;  >
----------------------------------------
----------------------------------------


Line:     int multiply = a*b;
Token detected: "int", "multiply", "=", "a", "*", "b", ";"
Findings : ----------------
Token Stream :
<  KEYWORD , int  >
<  IDENTIFIER , multiply  >
<  ASSIGNMENT_OP , =  >
<  IDENTIFIER , a  >
<  ARITHMETIC_OP , *  >
<  IDENTIFIER , b  >
<  DELIMITER , ;  >
----------------------------------------
----------------------------------------


Line:     total = 11.5*a + 21.22.;
Token detected: "total", "=", "11.5", "*", "a", "+", ".", ";"
Findings : ----------------
Token Stream :
<  IDENTIFIER , total  >
<  ASSIGNMENT_OP , =  >
<  FLOAT_CONST , 11.5  >
<  ARITHMETIC_OP , *  >
<  IDENTIFIER , a  >
<  ARITHMETIC_OP , +  >
<  DELIMITER , .  >
<  DELIMITER , ;  >
```

```
Error Findings : ----------------
Errors: [Invalid numeric format: 21.22]
----------------------------------------
----------------------------------------


Line:     vector<ll>a;
Token detected: "vector", "<", "ll", ">", "a", ";"
Findings : ----------------
Token Stream :
<  IDENTIFIER , vector  >
<  RELATIONAL_OP , <  >
<  IDENTIFIER , ll  >
<  RELATIONAL_OP , >  >
<  IDENTIFIER , a  >
<  DELIMITER , ;  >
----------------------------------------
----------------------------------------


Line:     for(ll i=0;i<a.size();i++)
Token detected: "for", "(", "ll", "i", "=", "0", ";", "i", "<",
     "a", ".", "size", "(", ")", ";", "i", "++", ")"
Findings : ----------------
Token Stream :
<  KEYWORD , for  >
<  DELIMITER , (  >
<  IDENTIFIER , ll  >
<  IDENTIFIER , i  >
<  ASSIGNMENT_OP , =  >
<  INT_CONST , 0  >
<  DELIMITER , ;  >
<  IDENTIFIER , i  >
<  RELATIONAL_OP , <  >
<  IDENTIFIER , a  >
<  DELIMITER , .  >
<  IDENTIFIER , size  >
<  DELIMITER , (  >
<  DELIMITER , )  >
<  DELIMITER , ;  >
<  IDENTIFIER , i  >
<  INCDEC_OP , ++  >
<  DELIMITER , )  >
----------------------------------------
----------------------------------------


Line:         cout << a[i] << ' ';
Token detected: "cout", "<<", "a", "[", "i", "]", "<<", "' '",
    ";"
```

```
Findings : ----------------
Token Stream :
<  IDENTIFIER , cout  >
<  BITWISE_OP , <<  >
<  IDENTIFIER , a  >
<  DELIMITER , [  >
<  IDENTIFIER , i  >
<  DELIMITER , ]  >
<  BITWISE_OP , <<  >
<  CHAR_CONST , ' '  >
<  DELIMITER , ;  >
----------------------------------------
----------------------------------------


Line:     cout << endl;
Token detected: "cout", "<<", "endl", ";"
Findings : ----------------
Token Stream :
<  IDENTIFIER , cout  >
<  BITWISE_OP , <<  >
<  IDENTIFIER , endl  >
<  DELIMITER , ;  >
----------------------------------------
----------------------------------------


Line: }
Token detected: "}"
Findings : ----------------
Token Stream :
<  DELIMITER , }  >
----------------------------------------
----------------------------------------
```

5.3 Output Token Statistics

The token statistics for the above input file is following:-

```

========== Summary ==========
Total Keywords: 8

Total Identifiers: 35

Total Constants(Integer): 1
Total Constants(Float): 1
```

```
Total Constants(String): 0
Total Constants(Character): 1

Total Operators(Arithmetic): 5
Total Operators(Relational): 3
Total Operators(Logical): 0
Total Operators(Bit-wise): 4
Total Operators(Assignment): 5
Total Operators(Inc/Dec): 1

Total Delimiters: 31

Total Tokens: 95

Unique Keywords: [for, int, namespace, using]

Unique Identifiers: [a, b, cin, cout, diff, endl, i, include,
    iostream, ll, main, multiply, size, std, summation, t, total
    , vector]

========== Error Summary ==========
Line 4: Unknown symbol: @
Line 4: Invalid identifier: 5pair
Line 9: Invalid numeric format: 21.22

========== Comments ==========
// initialize a vector
```

5.4 Output Symbol Table

The following symbol table is generated for the frequency of the identifiers:-

Table 1: Generated Symbol Table

Identifier	Count	First Line
a	8	4
b	4	4
cin	1	5
cout	2	13
diff	1	7
endl	1	14
i	4	12
include	1	1
iostream	1	1
ll	2	11
main	1	3
multiply	1	8
size	1	12
std	1	2
summation	1	6
t	2	4
total	2	4
vector	1	11

Here, in the above table all the identifiers with their frequency count in the input file along with their first line of occurence in the input file is shown.

5.5 Execution proof

The following figure shows a sample output of the lexical analyzer, by compiling the code representing the proof of execution.

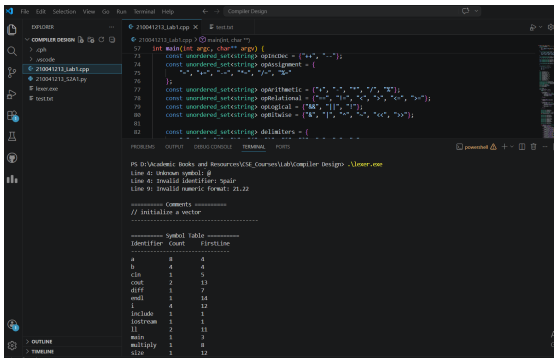

Figure 1: Sample Output of the Lexical Analyzer